

玲珑机械臂项目面试问答

EtherCAT 与 ros2_control 控制链路及 Linux 实时性

整理日期 2026 年 10 月 4 日

本资料汇总本次对话涉及的全部项目面试问题与关键追问，按候选人口吻组织答案，用于口头复习和技术追问准备。内容围绕四关节左臂的通信、硬件接口、驱动器使能、轨迹执行、故障处理与实时机制展开。

回答时应始终区分标准机制、项目代码已有实现，以及尚未完成现场验证的能力。代码中存在一个功能，不等于已经完成真机调试或性能验证。

项目事实与回答边界

- 当前讨论的真机插件是 LeftArmSystem，底层使用 IgH EtherCAT，统一管理 joint_1、joint_2、joint_3、joint_5 四个关节。CAN 断线问题按通用设计原则作答，不应说成当前真机插件使用 CAN。
- 驱动模式是 CSP。当前控制器配置为 100 Hz，对应名义周期 10 ms；不能声称项目已经完成 1 kHz 实时控制验证。
- 硬件暴露 position 命令接口和 position、velocity 状态接口，没有暴露 effort。
- 代码提供 DC 配置与同步调用，但配置模板 dc_assign_activate 为 0，不能声称当前已启用并验证高精度 DC 同步。
- 运行期反馈不完整会立即锁存整臂故障、关闭命令通路并尝试禁用驱动器。发送禁用命令不等于已经确认机械臂物理停止。
- 真机配置模板仍包含待标定参数和确认项。代码能力不应表述成已经完成所有现场标定、看门狗和抱闸验证。
- 没有系统做过完整 jitter benchmark，目前主要做到实时机制理解和项目环境使用。已有周期监测字段和模拟抖动测试不等于系统实时性基准测试。

阅读顺序

- 一 EtherCAT 基础与机器人控制 二 ros2_control 接口与生命周期
三 CiA402 使能状态机 四 CSP 与轨迹插补
五 EtherCAT 分布式时钟 六 Working Counter 与故障策略
七 Linux 实时调度与测量边界 附录 代码依据与技术参考

一 EtherCAT 基础与机器人控制

01 EtherCAT 和普通 Ethernet 最大区别是什么

最大的区别是通信的确定性和数据处理机制，而不只是速度。Ethernet 本身是一种以太网技术；通常说的普通以太网通信，是通过交换机和 TCP/IP 等协议传递数据，延迟会受到排队、协议处理和系统调度等因素影响，默认没有严格的周期保证。

EtherCAT 使用以太网帧，周期过程数据通常直接在二层传输。从站在帧经过时直接读取自己的命令、写入自己的反馈，不必每个设备分别完成一次请求与响应，因此适合确定周期的工业控制。

不能简单说 Ethernet 一定不能实时；更准确的说法是 EtherCAT 专门针对工业实时通信设计。[1]

02 为什么 EtherCAT 适合机器人控制

机器人多关节控制不仅要求数据及时到达，还要求多个关节在一致的时间基准下执行动作。EtherCAT 的周期交换效率高，适合持续交换多个轴的命令与反馈；延迟和抖动容易控制；分布式时钟 DC 支持多轴同步。[1]

我的项目通过 IgH 主站统一管理四关节左臂，采用 CSP 模式。不过当前配置为 100 Hz，DC 模板为 0，因此不能把协议的高性能能力直接当成项目已验证结果。系统实时性还取决于操作系统调度、控制线程、驱动与现场配置。

03 PDO 和 SDO 有什么区别

PDO 是过程数据对象，主要用于周期控制；SDO 是服务数据对象，主要用于参数配置和诊断。EtherCAT 的 CoE 提供这些对象字典访问机制。[1]

项目中，高频运动数据走 PDO，配置参数走 SDO；不在实时控制循环里进行阻塞式 SDO 操作。

- PDO 输出：目标位置 0x607A、控制字 0x6040。
- PDO 输入：实际位置 0x6064、实际速度 0x606C、状态字 0x6041、模式显示 0x6061。
- SDO 配置：配置阶段写入 0x6060 = 8，选择 CSP。
- RxPDO、TxPDO 从从站角度命名：RxPDO 是驱动器接收的命令，TxPDO 是驱动器发送的反馈。

二 ros2_control 接口与生命周期

04 ros2_control 解决什么问题

它解决控制算法与具体硬件驱动的解耦，以及控制资源和执行周期的统一管理。上层控制器使用标准关节接口，不需要知道底层是 EtherCAT、CAN 还是仿真硬件；硬件插件负责协议适配、单位换算和设备交互。

项目中，JointTrajectoryController 执行关节轨迹，LeftArmSystem 对接真实 EtherCAT 左臂，JointStateBroadcaster 发布关节状态，Controller Manager 组织 read → update → write 控制循环并管理控制器。

ros2_control 提供框架，但驱动器使能、通信检查、单位换算与故障保护仍需要硬件插件实现。

05 read 和 write 分别干什么

read() 把硬件实际状态读进来，write() 把控制器计算的命令送出去。每个周期先 read()，控制器再 update()，最后 write()。

我的 read() 接收 PDO，检查链路、工作计数器完整性和从站状态，再检查驱动器状态、模式和反馈位置等，将编码器数据换算为 rad、rad/s。

write() 从命令缓冲区读取目标位置，检查有限值、位置范围、单周期步长、命令变化速度和跟随误差，然后换算成编码器计数并通过 PDO 发送。

两者位于周期路径，不能无限等待通信恢复，不适合阻塞式配置、频繁分配内存或耗时操作。

06 command_interface 和 state_interface 有什么区别

区别是数据方向和含义。state_interface 是硬件向控制器提供的实际状态；command_interface 是控制器向硬件提供的期望值。

例如 joint_1/position 可以同时存在于状态接口和命令接口中，但分别表示实际位置与目标位置，应该绑定不同的内存。状态接口通常允许多个控制器或广播器读取；命令接口通常由一个控制器独占，避免多个控制器同时驱动同一执行量。

07 position velocity effort 怎么暴露

硬件描述、插件导出和控制器配置必须一致。先在 ros2_control 的关节描述中声明支持的接口，再由硬件插件导出接口并绑定生命周期足够长的成员变量，最后由控制器选择对应接口。

```
<joint name="joint_1">
  <command_interface name="position"/>
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>
```

```
StateInterface(name, "position", &exported_positions_[i]);
StateInterface(name, "velocity", &exported_velocities_[i]);
CommandInterface(name, "position", &commands_[i]);
```

转动关节的 position 单位为 rad，velocity 为 rad/s，effort 为 N·m。若底层反馈电流，必须结合电机与传动参数换算，不能把电流直接当成关节力矩。当前项目仅暴露 position 命令和 position、velocity 状态，没有 effort。增加速度或力矩控制还需实现驱动模式、协议映射、限幅和模式切换，不能只加接口名称。

08 Hardware Interface 生命周期是什么

标准主要状态是 UNCONFIGURED、INACTIVE、ACTIVE、FINALIZED，通过生命周期回调进行转换。[2]

- `on_init()`: 解析参数，检查关节与接口，初始化内部数据。
- `on_configure()`: 建立通信并完成配置，成功后进入 INACTIVE。
- `on_activate()`: 完成使能与启动准备，成功后进入 ACTIVE。
- `on_deactivate()`: 停止正常运动输出，回到 INACTIVE。
- `on_cleanup()`: 释放配置和通信资源，回到 UNCONFIGURED。
- `on_shutdown()`: 关闭并释放资源，进入 FINALIZED。
- `on_error()`: 处理错误。

项目另有 DISCOVERING、CONFIGURING、ACTIVATING、FAULT 等内部状态，用来描述更细的硬件过程；它们不是 `ros2_control` 的标准生命周期状态。实际命令门控必须在插件中实现，不能只依赖状态名称。

09 为什么 configure 和 activate 要分开

因为建立通信和允许执行器运动是两个不同阶段。`configure` 阶段检查设备身份、PDO 映射、模式、反馈和看门狗，同时保持驱动器未使能；`activate` 阶段才推进驱动器使能状态机，并读取实际反馈确认结果。

项目启动时把目标位置对齐实际位置，避免使能瞬间跳向默认零点。硬件 ACTIVE 后也不立即接受旧目标：控制器取得四个关节的命令接口时，再重新对齐命令并打开 `commands_enabled_`。硬件就绪与控制器接管同样分开。

10 如果 CAN 断了 read 应该怎么办

当前真机插件使用 EtherCAT；如果换成 CAN，我会采用相同原则：识别有效反馈是否失效，禁止继续输出运动命令，锁存故障，并向框架返回错误。

- 区分暂时无帧与真正断线。非阻塞接收暂时没有数据不等于断线，要结合每轴最后有效反馈时间、心跳和 bus-off 等状态判断。
- 达到故障条件后不伪造反馈。不把命令值当实际位置，不把旧数据当新反馈；保留诊断值时必须标记失效。
- 关闭命令通路并返回 ERROR。周期线程不能无限重连。Jazzy 中 `read()/write()` 返回 ERROR 会触发 `on_error()`。[3]
- 总线断开后停止命令可能送不到，必须依靠预先配置并验证的驱动器通信看门狗和停机、抱闸行为。

当前 EtherCAT 实现中，周期反馈不完整会锁存故障、关闭命令、尝试禁用输出，将对外位置和速度置为 NaN，并返回 ERROR。故障不会因通信恢复自动清除，`cleanup` 后仍保留，需要排除原因并重启。返回 ERROR 是通知框架；物理停机需要命令门控和驱动器失联保护共同完成。

三 CiA402 使能状态机

11 CiA402 状态机怎么让电机 Enable

核心是主站写 Controlword 请求状态转换，再读 Statusword 确认驱动器实际进入目标状态。不能连续发送 6、7、F 就认为使能成功。

从 Switch On Disabled 出发，典型路径是 Ready to Switch On → Switched On → Operation Enabled，对应命令 Shutdown → Switch On → Enable Operation。[4]

12 Controlword 0x6040 是什么

它是对象字典中的 16 位控制字，由主站写给驱动器，用于请求使能、去使能、快速停止和故障复位等操作。0x6040 是对象索引，0x0006、0x0007、0x000F 是写入值。

- bit 0: Switch On。bit 1: Enable Voltage。
- bit 2: Quick Stop；正常使能序列中置 1，表示不请求快速停止。
- bit 3: Enable Operation。bit 7: Fault Reset。

Shutdown = 0x0006 = 0000 0110
Switch On = 0x0007 = 0000 0111
Enable Operation = 0x000F = 0000 1111

Shutdown 是使驱动器进入 Ready to Switch On 的状态机命令名称，不是关闭整个应用程序。

13 Statusword 0x6041 是什么

它是驱动器反馈的 16 位状态字，反映实际状态，包括是否准备好、是否使能、是否故障等。Controlword 表示我要什么状态，Statusword 表示驱动器实际是什么状态。

项目用 $(\text{statusword} \& 0x006F) == 0x0027$ 判断 Operation Enabled。必须先做掩码再比较，不能直接要求整个状态字等于 0x0027，因为警告、厂家自定义和模式相关位可能同时存在。

14 Shutdown 到 Switch On 再到 Enable Operation 怎么走

每一步都根据新反馈决定下一步，并设置启动超时，不能靠固定 sleep 猜测状态已经完成转换。

Switch On Disabled : $(\text{sw} \& 0x004F) == 0x0040$

写 0x0006 → Ready to Switch On

Ready to Switch On : $(\text{sw} \& 0x006F) == 0x0021$

写 0x0007 → Switched On

Switched On : $(\text{sw} \& 0x006F) == 0x0023$

写 0x000F → Operation Enabled

Operation Enabled : $(\text{sw} \& 0x006F) == 0x0027$

正常运行时继续保持 0x000F。项目推进序列的前提还包括完整有效反馈、没有锁存故障和运行模式正确。

15 项目里怎样才算 Enable 成功

四轴反馈完整，四个驱动器均满足 $(\text{statusword} \& 0x006F) == 0x0027$ ，且模式显示 0x6061 均为 8，core_→activate() 才成功。启动期间目标位置对齐实际位置，控制器接管后才接受新运动目标。

进入 Fault 后项目不会自动反复发送 Fault Reset。标准故障复位涉及 bit 7 上升沿，但应先排除原因，再执行明确恢复流程。

EtherCAT OP 表示正常过程数据通信状态；CiA402 Operation Enabled 表示驱动器已使能。总线 OP 不等于电机 Enable。

四 CSP 与轨迹插补

16 CSP 是什么

CSP 是 Cyclic Synchronous Position，即周期同步位置模式，CiA402 模式编号为 8。主站按固定周期发送当前时刻的位置目标，驱动器执行位置跟踪，内部完成位置、速度和电流闭环。[5]

项目通过 $0x6060 = 8$ 设置 CSP，检查 $0x6061 == 8$ 确认，再通过 PDO 周期发送 $0x607A$ 。核心是持续下发轨迹上的位置采样点，不是只发送一个终点。

17 CSP 和 Profile Position 有什么区别

主要区别是运动轨迹由谁生成。CSP 模式 8 由主站生成各时刻位置目标并周期下发；Profile Position 模式 1 则由主站发送目标位置及轮廓速度、加减速参数，由驱动器内部生成运动轮廓，通常通过新设定点握手触发执行。[4][5]

例如转到 1 rad：PP 是告诉驱动器按指定速度和加速度走到 1 rad；CSP 是持续告诉驱动器这一时刻到哪里、下一时刻到哪里。PP 也能更新目标，但 CSP 更方便多轴按照统一时间轴协调执行。

18 Target Position 多久更新一次

正常运行时每个控制周期更新并发送一次，CSP 本身没有规定必须为 1 ms。项目 `update_rate` 为 100 Hz，所以名义周期为 10 ms，每周期执行 `read` → `controller.update` → `write`。

保持位置时目标数值可以不变，但周期通信仍继续。上层轨迹消息频率不等于目标位置更新频率：上层可以一次提交持续数秒的轨迹，控制器每 10 ms 从中计算一次目标。

19 插补在哪里做

项目中，轨迹点之间的时间插补主要由主机侧 `JointTrajectoryController` 完成。硬件 `write()` 负责检查、单位换算与 PDO 发送，不承担整条轨迹规划。

上层带时间的轨迹点

- JTC 按当前时间插补和采样
- `position command_interface`
- `write` 检查与换算
- PDO
- 驱动器位置跟踪

Jazzy JTC 默认采用 spline 插补，阶次取决于输入字段：只有位置时为线性；位置和速度为三次；位置、速度和加速度为五次。即使硬件仅暴露 `position` 命令接口，轨迹中的速度和加速度仍可参与主机插补，不必直接发送给驱动器。[6]

驱动器内部伺服周期通常比通信周期短，可能对设定点继续平滑或细分插补。因此不能说 CSP 驱动器内部绝对没有插补。当前代码可确认主机侧 JTC 插补，不能据此断言 EYOU 驱动器内部具体采用哪种算法。

五 EtherCAT 分布式时钟

20 EtherCAT DC 解决什么问题

DC 是 Distributed Clocks，即分布式时钟，让不同从站拥有一致时间基准，从而在约定时刻采样和执行。即使命令放在同一帧中，经过各从站仍有先后，各驱动器本地时钟也存在偏差和漂移。同一帧发送不代表同时执行。

DC 对齐各从站时钟，再通过配置的 SYNC0、SYNC1 等硬件同步事件统一周期和相位，降低报文到达抖动对执行时刻的影响。但它不会消除主机调度延迟：主站仍必须在目标被使用前及时送达正确数据。[1][7]

21 主站时钟和从站时钟怎么同步

应区分从站之间同步与主站对齐总线时间。从站同步包括选择参考时钟、测量传播延迟、补偿时钟偏移，以及周期校正晶振漂移。参考时钟通常是第一个支持 DC 的从站，也可指定；不一定是主机系统时钟。

主站可以跟随参考从站，也可以让参考从站跟随主站应用时间。项目 IGH 代码预留的是后者：[7][8]

```
ecrt_master_application_time(master_, application_time);
ecrt_master_sync_reference_clock(master_);
ecrt_master_sync_slave_clocks(master_);
```

依次提供应用时间、组织参考时钟向应用时间同步、组织其他从站向参考时钟同步；同步报文随发送过程发往总线，并非函数返回时所有时钟就瞬间对齐。项目用 `ecrt_slave_config_dc()` 设置参数，应用时间来自 `steady_clock`。

模板 `dc_assign_activate` 为 0，实际同步调用受非零配置条件控制。因此当前应说代码支持 DC 接入，不能说已经启用并验证同步精度。

22 为什么多个关节不同步会有问题

末端轨迹取决于所有关节在同一时刻的组合位置。规划需要 $q_1(t)$ 、 $q_2(t)$ 、 $q_3(t)$ 、 $q_5(t)$ ，若某轴执行 $q_2(t-\Delta t)$ ，组合姿态就偏离轨迹。即使各轴最终到位，运动过程仍可能走偏。

后果包括末端轮廓误差、把不同时刻反馈混在一起造成状态估计误差，以及装配或双臂协作中的额外内力、冲击与振动。小时间偏差下，关节位置偏差约等于关节速度乘时间偏差，所以速度越高越敏感。

JTC 负责让四轴目标属于同一时间轴，DC 负责对齐驱动器采样与执行时刻，两者共同支持多轴协调。

六 Working Counter 与故障策略

23 Working Counter 是什么

WKC 是 EtherCAT 数据报中的 16 位工作计数器，用于检查预期从站是否成功处理本次读写操作。从站硬件在成功操作后增加计数，主站比较实际值与预期值。它不是简单的在线从站数量。[9]

- 单独读命令：成功读加 1。单独写命令：成功写加 1。
- 读写命令如 LRW：读成功加 1，写成功加 2，两者均成功加 3。
- 预期值取决于命令和过程数据映射，四个从站不代表预期 WKC 必然等于 4。

24 WKC 异常说明什么

说明本次过程数据交换没有按预期完整完成，不能把整组反馈当作有效新数据。原因可能包括从站掉电、链路异常、退出预期状态、地址或映射配置错误，以及报文丢失、超时或尚未完整返回。

仅靠 WKC 不能直接定位具体关节和原因，还要结合链路、从站状态、接收超时与错误计数。反过来，WKC 正常也不等于电机已使能、目标已执行、编码器合理或 DC 已同步；项目还检查状态字、模式和跟随误差。

25 项目怎么检查 WKC

项目使用 IgH domain 状态，要求链路正常且 `wc_state` 为 `EC_WC_COMPLETE`，再要求四个从站都 online 且 operational。IgH 还定义 `EC_WC_ZERO` 和 `EC_WC_INCOMPLETE`。[10]

```
bool complete = ms.link_up && ds.wc_state == EC_WC_COMPLETE;  
// 随后还检查每个从站 ss.online && ss.operational
```

complete 是链路、WKC 与从站状态的综合结果，不能把所有 `complete == false` 都一律记录为 WKC 错误。

26 一次 WKC 异常会立刻停机吗

结合当前项目，正常运行期间会立即触发整臂故障停机流程。`core_>feedback()` 遇到 `!complete` 会 `fail(6)`，随后锁存故障、禁止运动命令、尝试禁用输出，并让 `read()` 返回 `ERROR`。

四轴协同运动时，只要一轴反馈失去可信度，就不能假设整臂状态有效。当前没有连续丢若干帧才停机的容错策略。启动阶段不同：首次完整反馈之前允许在启动超时内等待；已经收到完整反馈后再丢失就故障。

立即触发停机并不保证物理上立即停止，总线异常时禁用命令可能无法送达，仍需已验证的通信看门狗和停机、抱闸响应。

一般工程策略可以允许经验证的短暂容错，但阈值应依据风险、速度、驱动器行为和最大反馈失效时间，不能随意说丢三帧没关系。单次异常不必然代表永久断线；本项目选择运行期单次异常锁存故障。

七 Linux 实时调度与测量边界

27 SCHED_FIFO 是什么

SCHED_FIFO 是 Linux 固定优先级抢占式实时调度策略。FIFO 实时优先级通常为 1 至 99，数值越大优先级越高，与普通进程 nice 值不同。

同优先级按队列规则调度，没有普通时间片轮转。线程通常持续运行，直到阻塞、主动让出或被更高优先级线程抢占。[11]

28 priority 越高越好吗

不是。要根据控制周期、执行时间与整条控制链依赖分配优先级，而不是直接设为 99。

例如 PREEMPT_RT 下很多中断处理在线程中执行；若控制线程优先级高于它依赖的网卡 IRQ 线程，又一直忙等，网卡处理可能被饿死，反而拿不到反馈。高优先级不能补偿错误的阻塞与依赖设计。

29 如果实时线程永不 sleep 会怎样

不调用 sleep 不一定一直占用 CPU，也可能通过等待事件或 I/O 阻塞。但如果 FIFO 线程始终可运行且不让出，就会持续占用所在 CPU，低优先级线程可能被饿死，同优先级线程也不会因时间片耗尽自动获得机会。

它仍可能被更高优先级线程、中断打断，Linux 实时带宽限制也可能节流，具体看配置。周期控制应采用基于单调时钟的绝对时间等待，按计划周期唤醒，而不是工作完再 sleep 10 ms，以免执行时间累积进入周期。[11]

30 mlockall 干什么

mlockall() 把进程内存页锁在 RAM 中，避免换出和重新调入造成不可预测延迟。常用 mlockall(MCL_CURRENT | MCL_FUTURE)：前者锁定当前映射，后者使后续新增映射也受锁定约束。

必须检查返回值和锁内存额度。它不保证所有内存操作无延迟，因此实时循环前还应预分配缓冲区、预触碰栈和工作内存，避免循环内动态分配。[12]

31 page fault 为什么影响实时性

Page fault 是访问虚拟内存时，当前映射或权限条件无法直接满足访问，需要内核介入。Minor fault 通常无磁盘 I/O，但可能涉及映射、页面分配或写时复制；Major fault 需要存储设备读取，延迟可能更大。

原本一次内存访问变成耗时不稳定的内核处理，就可能错过控制截止时间。锁内存与预触碰是减少不确定性，不是让计算本身变快。

32 CPU affinity 为什么有用

CPU affinity 限制线程能在哪些 CPU 上运行。绑核减少迁移造成的缓存失效与时间波动，配合其他任务分配后可减少无关负载竞争。

绑核只是限制运行位置，不等于独占。其他线程、IRQ 和内核任务仍可能运行在同一 CPU 上。

33 IRQ 在哪个 CPU

IRQ 有独立的亲和性，不能从控制线程绑核位置推断。应在目标机器查看每 CPU 中断计数、允许亲和性与实际有效亲和性：[13]

```
cat /proc/interrupts
cat /proc/irq/<IRQ 号>/smp_affinity_list
cat /proc/irq/<IRQ 号>/effective_affinity_list
```

还要考虑 irqbalance、多队列网卡和内核管理的中断。我目前没有目标机器 IRQ 分布证据，不能声称 EtherCAT 网卡 IRQ 已固定在某 CPU。如果驱动使用轮询，还应结合具体驱动核实收发路径。

34 实时线程绑核后就一定实时了吗

不一定。实时性关注能否在截止时间内完成，不是是否绑核。还要考虑内核抢占、IRQ 干扰、最坏执行时间、锁竞争和优先级反转、阻塞 I/O、缺页、内存与缓存竞争、节能唤醒和固件延迟，以及通信是否及时到达。

FIFO、绑核、锁内存都是降低延迟波动的措施。是否满足实时要求必须通过测量验证。

35 你实际测过 jitter 吗

没有系统做过完整 jitter benchmark，目前主要做到实时机制理解和项目环境使用。

项目已有 period_seconds、max_period_seconds、deadline_misses 字段，但主要记录控制调用间隔。

deadline_misses 按间隔超过名义周期 1.5 倍计数，并非严格测量每次唤醒相对计划截止时间的偏差。测试里的随机 jitter 是人为注入输入，不能当成真实调度抖动测量结果。

后续完整验证应分别测线程唤醒延迟、循环执行时间和端到端通信时序，在持续负载下统计分布、最大值与超期次数。目前不能给出已验证的微秒级 jitter 指标。

附录 代码依据与技术参考

以下路径相对于项目根目录 C:/Users/Administrator/Desktop/linglong_1025_2。代码依据用于解释实现，不替代目标硬件现场验证。

硬件接口与生命周期

ros2_ws/src/linglong_control/src/left_arm_system.cpp

IgH 总线与 PDO 和 DC

ros2_ws/src/linglong_control/include/linglong_control/left_arm_bus.hpp

使能状态机与命令保护

ros2_ws/src/linglong_control/include/linglong_control/left_arm_core.hpp

内部硬件状态和健康字段

ros2_ws/src/linglong_control/include/linglong_control/hardware_state.hpp

控制周期和接口配置

ros2_ws/src/linglong_control/config/controllers.yaml

真机配置模板与待确认参数

ros2_ws/src/linglong_control/config/left_arm_hardware.yaml

仿真关节接口声明

ros2_ws/src/linglong_control/urdf/arm_control.urdf.xacro

真机参数转换

ros2_ws/src/linglong_control/linglong_control_tools/left_arm_configuration.py

模拟抖动测试

ros2_ws/src/linglong_control/test/sim_validation.cpp

技术参考

- [1] EtherCAT Technology Group 技术说明[阅读原文](#)
- [2] ros2_control Jazzy 硬件生命周期[阅读原文](#)
- [3] ros2_control Jazzy read 和 write 错误处理[阅读原文](#)
- [4] Synapticon CiA402 状态机与模式[阅读原文](#)
- [5] Synapticon CSP 说明[阅读原文](#)
- [6] JTC Jazzy 轨迹表示与插补[阅读原文](#)
- [7] ETG 分布式时钟说明[阅读原文](#)
- [8] IgH EtherCAT Master 应用接口[阅读原文](#)
- [9] Beckhoff Working Counter [阅读原文](#)
- [10] IgH domain 状态与 WKC [阅读原文](#)
- [11] Linux sched 调度手册[阅读原文](#)
- [12] Linux mlock 与 mlockall 手册[阅读原文](#)
- [13] Linux SMP IRQ affinity [阅读原文](#)